



AYURSAGE (An AI-powered Ayurvedic Advisory System)

Final End-Semester Project Presentation

for

BACHELOR OF TECHNOLOGY

Degree in

COMPUTER SCIENCE AND ENGINEERING

By

GROUP NO:-27

Aditi Verma	2200640100008
Diksha Sharma	2200640100051
Jagveer Singh Bedi	2200640100075
Yashi Upadhyay	2200640100171

Under the Guidance of

Guide Name (Ms. Ayushi Sharma)

Designation (Assistant Professor)

Department of Computer Science and Engineering

Hindustan College of Science and Technology, Farah, Mathura

DR. A. P. J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW,

INDIA

Academic Session 2025-26

Table of Content

Sr. No	Topic	Slide No.
1	Project Overview	3
2	Objectives of Project	4
3	Literature Review (Previous research work)	5 - 8
4	System Design	9
5	Dataset Development	10-14
6	Coding and Implementation Details	15-32
7	Result	33
8	Project Timeline and Future Work	34
9	Conclusion	35

1. Project Overview

- **Ayurveda** is a 5000+ year-old holistic medical system from India.
- Health is based on the **balance of three Doshas** – Vata, Pitta, and Kapha.
- **Problem:**
 - Traditional Ayurvedic consultation is time-consuming and depends heavily on practitioner expertise.
 - Data unavailability
- **Need:** A scalable, AI-powered solution to make Ayurvedic knowledge **accessible and personalized**.
- **Proposed Solution:** *AyurSage* – a web-based consultation system using **Machine Learning models** to bridge the gap by developing a symptom-to-dosha mapping system, predicting possible diseases, and providing personalized Ayurvedic solutions using AI techniques.

2. Objectives of Project

- 1) To build a system that identifies Ayurvedic **dosha imbalance** from user-reported symptoms using machine learning.
- 2) To create a pipeline that links **symptoms → dosha → disease → remedy** for holistic diagnosis.
- 3) To recommend **personalized Ayurvedic treatments** (medicines, diet, lifestyle) based on predicted dosha/disease.
- 4) To design an **explainable and user-friendly platform** that integrates traditional Ayurvedic wisdom with modern data science.
- 5) To evaluate the system's **accuracy and practical usefulness** through expert validation and user feedback.

3. Literature Review (Previous research work)

S. No.	Author(s) & Year	Title of Paper / Study	Objective / Purpose	Methodology / Techniques Used	Dataset / Tools Used	Key Findings / Results	Limitations / Gaps Identified	Relevance to Present Work
1	Jadhav et al., 2024	<i>Integration of Machine Learning in Ayurveda: An Indian Traditional Health Science</i>	To explore how ML techniques can be integrated into Ayurvedic healthcare for improved diagnosis, treatment personalization, disease prediction & public health monitoring.	Review of ML methods (Supervised, Unsupervised, Semi-supervised, Reinforcement ML) and classification algorithms (SVM, KNN, RF, NB, NN, LSTM).	Literature review; Ayurvedic texts (Samhitas); general ML algorithm frameworks.	ML can enhance Ayurvedic diagnosis accuracy, personalized treatment, medicine authentication & surveillance.	No experimental model; lacks clinical validation; no dataset-based results.	Establishes foundational justification for applying ML in Ayurveda; supports the conceptual framework of the present work.
2	Roy et al., 2025	<i>Bridging Psychometrics and Ayurveda Using Explainable Machine Learning</i>	To predict Ayurvedic Prakriti (Vata, Pitta, Kapha) through psychometric factors and provide explainable ML interpretations.	Class-weighted Random Forest , XGBoost, SVM, Logistic Regression, MLP; SHAP for explanation of feature influence.	Dataset of n=373 questionnaire responses; Tools: Scikit-learn, SHAP.	Best model: Class-weighted Random Forest (AUROC = 0.762). Found psychological correlations e.g., EQ → Vata, IQ → Kapha, Risk-taking → Pitta .	Dataset was self-reported; imbalanced classes; no clinical diagnosis confirmation.	Supports integrating explainable ML and behavioral features into Ayurveda-based predictive models in current research.
3	Sudha et al., 2025	<i>Bridging Ayurveda and AI: Data Standardization for Improved Machine Learning Application</i>	To standardize Ayurvedic terminology and dataset structure for accurate AI/ML model training and interoperability.	Text preprocessing, NLP-based Named Entity Recognition, Multi-label text classification using CNN , Naïve Bayes, BERT ; Ontology Mapping.	Ayurvedic formulation dataset from Kaggle ; Tools: SpaCy, Keras, BERT-base model.	BERT achieved 100% accuracy after standardization; reduced ambiguous/unknown terms significantly; improved dataset clarity.	Small dataset; Sanskrit compound words still challenging; clinical deployment not tested.	Shows the importance of structured & standardized datasets before ML model development; applicable to dataset preparation in proposed work.

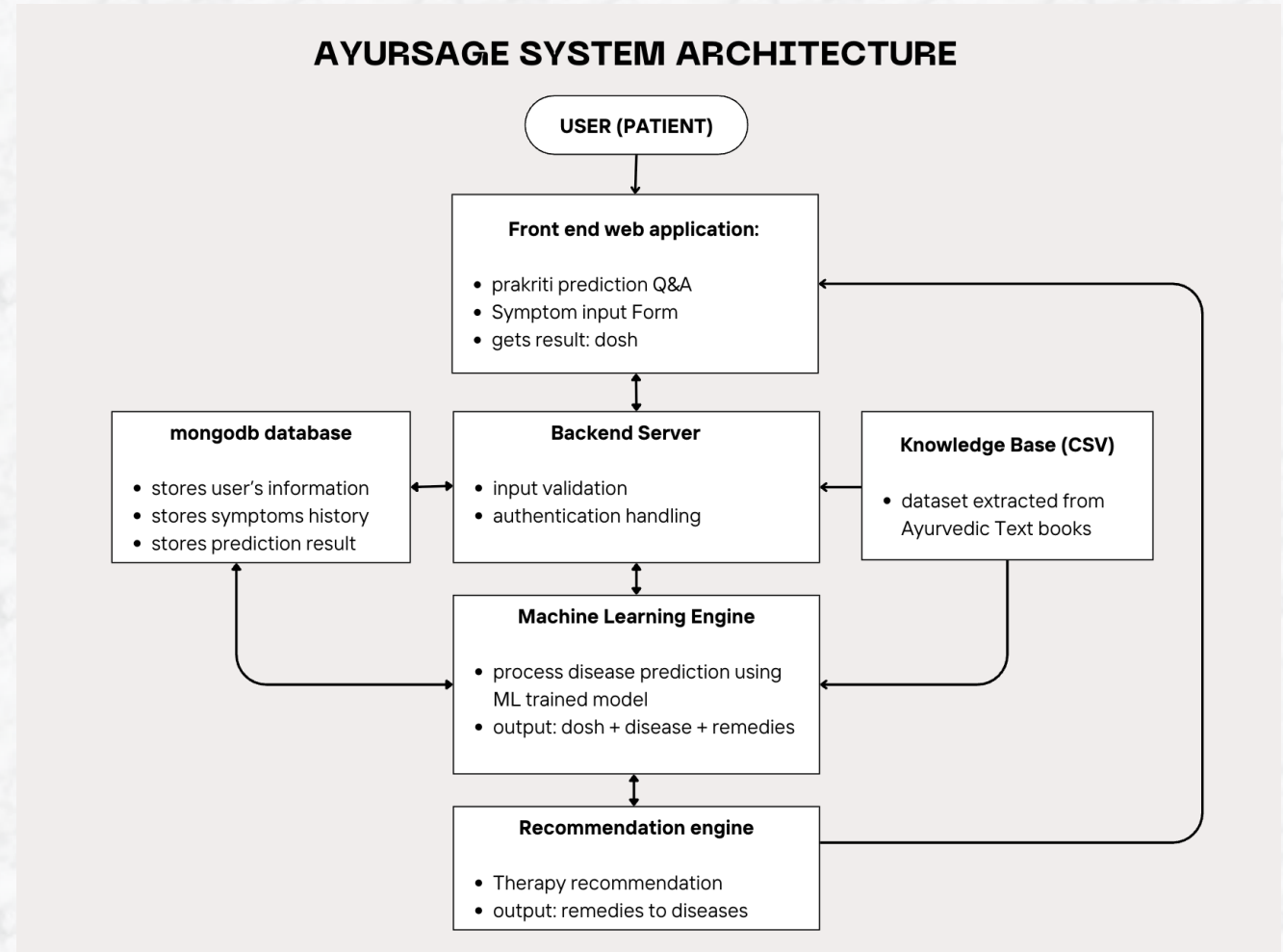
S. No.	Author(s) & Year	Title of Paper / Study	Objective / Purpose	Methodology / Techniques Used	Dataset / Tools Used	Key Findings / Results	Limitations / Gaps Identified	Relevance to Present Work
4.	Sharma, P. (2025)	Uses and Relevance of Artificial Intelligence (AI) in Ayurveda	To analyse how AI technologies can modernize Ayurveda by improving diagnosis, treatment personalization, research, and healthcare accessibility.	Qualitative narrative review using classical Ayurvedic texts and scientific publications from major databases (Google Scholar, PubMed, Scopus, Elsevier).	Ayurgenomics platforms, NLP tools, data mining algorithms, and general machine learning frameworks.	Highlights AI’s role in prakriti identification, ageing prediction, decoding ancient texts, virtual consultations, and supporting herbal drug discovery; shows strong potential for enhancing accuracy and patient outcomes.	Study lacks empirical testing; limited standardized datasets; ethical and privacy challenges; insufficient government-supported AI–Ayurveda integration models.	Serves as a foundational conceptual study for building AI-driven Ayurvedic diagnostic tools, personalized recommendation systems, and evidence-based digital Ayurveda platforms.
5.	Ramesh, V. (2025)	Applications of Machine Learning in Ayurinformatic Research	To examine how ML can support modernization of Ayurveda through computational analysis, predictive modeling, and data-driven insights.	Descriptive literature review covering ML concepts, use cases such as personalized medicine, NLP, herb–drug modeling, and predictive analytics.	Tools such as MySQL, PostgreSQL, Hadoop, Spark, NLTK, spaCy, TensorFlow, Keras, PyTorch, XGBoost, Tableau, Protégé, AWS, MATLAB; Ayurvedic sources like API and HerbMed.	ML supports personalized treatments, disease prediction, herb–drug interaction analysis, text mining, and decision support, promoting evidence-based Ayurveda.	Limited empirical validation; lack of standardized datasets; challenges in Ayurvedic terminology mapping; ethical and data privacy concerns.	Offers a strong technical base for developing AI/ML-driven Ayurvedic diagnostic tools, predictive systems, and digital health platforms.
6.	Gupta, R., & Mehta, P. (2025)	Artificial Intelligence in Ayurveda: A Systematic Review (2020–2025)	To review and evaluate AI–Ayurveda research from 2020–2025, focusing on applications, methods, and progress.	Systematic review using PRISMA; analysis of articles from PubMed, Scopus, IEEE Xplore, and Google Scholar; qualitative synthesis of AI applications in Ayurvedic diagnosis and treatment.	Data from 60+ studies; AI tools such as ML, DL, NLP, image processing; sources like AYUSH Research Portal, TKDL, and Ayurvedic Pharmacopoeia.	AI widely applied in Prakriti assessment, Dosha prediction, herbal drug discovery, diagnosis, and text mining; improves accuracy, personalization, and scientific validation.	Lack of standardized datasets; weak integration with classical Ayurvedic concepts; challenges in Sanskrit NLP; limited clinical validation.	Offers a comprehensive overview of current AI–Ayurveda trends and guides development of diagnostic, predictive, and intelligent Ayurvedic systems.

S. No.	Author(s) & Year (APA)	Title of Paper / Study	Objective / Purpose	Methodology / Techniques Used	Dataset / Tools Used	Key Findings / Results	Limitations / Gaps Identified	Relevance to Present Work
7.	Sudha et al., 2025	Bridging Ayurveda and AI: Data Standardization for Improved Machine Learning Application	Propose a data-standardization framework (ontology + NLP pipeline) for converting heterogeneous Ayurvedic text into machine-readable form to improve ML performance.	Preprocessing (tokenization, lemmatization), NER using SpaCy + transformer, multi-label text classification (spaCy textcat_multilabel), ontology mapping (TF-IDF + cosine similarity), ML models: Naïve Bayes, CNN, BERT (fine-tuned).	Kaggle dataset “Ayurvedic Formulations” (~300 entries); tools: SpaCy, Keras, transformers; evaluation via accuracy, precision, recall, F1.	Standardization improved accuracy; BERT achieved 100% , CNN ~80%, NB ~50%. UNKNOWN labels reduced from 287 → 0. Ontology + transformers highly effective for Ayurvedic ML.	Small dataset (~300), possible incorrect mappings, overfitting risk due to very high BERT accuracy, limited domain-specific validation, no clinical testing.	Useful for building ontologies, handling Ayurvedic heterogeneity, and benchmarking NLP/ML models in similar projects.
8.	Gupta et al., 2025	Artificial Intelligence (AI) in Ayurveda: Its Application and Relevance	Review AI use-cases in Ayurveda—diagnostics, Ayurgenomics, herb ID, formulation optimization, quality control, and knowledge retrieval.	Narrative literature review surveying classical texts and recent research; discusses AI techniques (NLP, image recognition, ML, molecular modelling).	Literature-based review; mentions tools (NLP, wearables, spectral imaging), no primary dataset.	Highlights AI potential: Prakriti-genomics, pulse/tongue imaging, herb ID (image/spectral), personalized preventive care, quality control.	Identifies data scarcity, oversimplification of Ayurvedic concepts, privacy/ethical issues, lack of regulation and standardized datasets.	Good for background and justification; outlines opportunities and risks, supports framing problem statements and ethical considerations.
9.	Majhi et al., 2023	ML-based Parkinson’s Disease Prediction Using Ayurvedic Dosha Analysis	Build ML models predicting Parkinson’s Disease by mapping clinical scales to Ayurvedic dosha scores and using them as features.	Data preprocessing in SPSS; dosha score derivation from UPDRS-II & NMSQ; ML models: LR, KNN, SVM, KSVM, NB, DT, RF, XGBoost; 10-fold cross-validation; metrics: accuracy, ROC/AUC, F1, MCC.	Fox Insight dataset (n=80,916); tools: SPSS v28, scikit-learn.	KSVM accuracy ~93%; LR selected as optimal (accuracy ~92.6%, lowest FPR ~0.0405). Dosha scores (especially Vata) correlated with PD likelihood.	No deep-learning models tested; missing features (family history, environment); Prakriti data absent; scale→dosha mapping may require clinical validation.	Relevant for combining clinical data with Ayurvedic constructs, deriving dosha-based features, and using structured datasets for disease prediction.

S. No.	Author(s) & Year	Title of Paper / Study	Objective / Purpose	Methodology / Techniques Used	Dataset / Tools Used	Key Findings / Results	Limitations / Gaps Identified	Relevance to Present Work
10.	Prof. S.S. Katkar et al. (2025)	Ayurveda Hub: Automated Prakruti Detection and Ayurvedic Reference Integration	To build an AI-based platform for Prakruti detection and Ayurvedic text search .	Used ML algorithms for Prakruti classification and NLP for text search. Developed using MERN Stack with models like Decision Tree, SVM, Neural Networks , and TF-IDF/cosine similarity.	User assessment data; digitized Ayurvedic texts (60+). Tools: ML models, NLP, MERN Stack.	Accurately identifies Prakruti with confidence score and gives lifestyle suggestions. Provides instant Ayurvedic text references.	Limited dataset variation; slow search on large data; challenges with mixed Prakruti types.	Relevant for developing AI-based Ayurveda diagnosis and digital knowledge systems.
11.	Pravit Akarasereenont & Ketmanee Jongjiamdee (2025)	<i>AI in Traditional Medicine: Evidence, Barriers, and Research Roadmap</i>	To review AI applications in Ayurveda, TCM, TTM and propose a research roadmap.	Narrative Mini Review using PubMed, Scopus, Google Scholar (2017–2025). Focus on diagnosis, personalization, herbal research, telemedicine.	Secondary data. Tools: ML, DL, NLP, CV , telemedicine systems; supporting TM–AI studies.	AI improves diagnosis (tongue, pulse, face), personalization, herbal analysis, telemedicine, and digital preservation.	Lack of standard datasets, limited validation, interpretability issues, weak regulations.	Helps guide future AI–Ayurveda integration , diagnostic system development, and TM modernization.
12.	Dr. Manjiri Ranade (2024)	<i>AI in Ayurveda: Concepts and Prospects</i>	To study how AI can support Ayurveda in diagnosis, treatment, and drug discovery.	Review of existing AI–Ayurveda research.	Databases: PubMed, Google Scholar, Scopus, Web of Science; 110 papers → 34 considered.	AI improves Prakriti prediction, personalized care, herbal discovery, predictive health, and virtual assistance.	Lack of datasets, limited validation, ethical/privacy concerns, small-scale studies.	Provides base for AI–Ayurveda tools and supports digital, personalized Ayurvedic systems.

4. System Design

- **Data Collection & Preprocessing**
- Extract symptoms & cures from Ayurvedic resources
- Source data : From classical Ayurvedic texts (Charaka Samhita, Sushruta Samhita, Ashtang Hridaya) and modern Ayurvedic references.
- Normalize symptom terms
- **Dosha Classification Model**
- ML algorithms (Random Forest)
- **Disease Prediction Model**
- Map dosha + symptoms → disease
- **Cure Recommendation**
- Gives remedies based on diseases



5. Dataset Development

Stage	Data Type	Dataset Features	Sources
Base Dataset	User profile & lifestyle factors Symptoms	Age, Gender, Stress, Sleep, Diet, Season	Indian national family health survey
Symptom → Dosha	Symptom lists & dosha imbalance mapping	Labelled Dosha	Sushruta Samhita Charaka Samhita Ashtanga Hriday
Symptom+ Dosha → Disease	Dosha-based disease classification	Most Likely Disease & Disease description.	Derived by matching textual symptoms and labelled Dosha in the texts to most likely Disease and disease description
Dosha + Disease → Cure	Remedies (herbal, diet, lifestyle, yoga)	Cure Recommendations (Therapy,medicines,exercise, Diet Plan)	Herbal Medicines & Diet Plan Lifestyle + Exercise/Yoga Guidance: Ashtanga Hridaya,

5. Dataset Development

Dataset Schema:

ID	Age	Gender	Prakriti	Symptoms	Stress Level	Sleep Pattern	Diet Type	Season	Climate	Dosha	Disease	Therapy	Medicines	Diet Plan	Exercise
1	49	Female	Kapha	sweet taste in mouth, lethargy, sv	High	Hypersomnia	Vegetarian	Varsha	Cold	Kapha	Prameha -	Virechana, Nasya	Trikatu, Guggulu	Warm spices	Brisk walking
2	31	Female	Pitta-Kapha	excessive urination, heaviness, tur	High	Interrupted	Vegetarian	Varsha	Humid	Kapha	Prameha -	Virechana, Udwarthanam	Trikatu, Jambu seeds	Light, dry food	Running
3	37	Female	Vata-Kapha	excessive urination, heaviness, tur	Low	Hypersomnia	Non-veget	Hemant	Humid	Kapha	Prameha -	Udwarthanam, Virechana	Guggulu, Jambu seeds	Light, dry food	Surya Namaskar
4	33	Male	Kapha	loss of appetite, excessive urinati	Moderate	Interrupted	Non-veget	Varsha	Hot	Kapha	Prameha -	Virechana, Nasya	Guggulu, Trikatu	Avoid sugar	Surya Namaskar
5	26	Female	Vata-Kapha	lethargy, weight gain, loss of appe	High	Interrupted	Vegetarian	Varsha	Cold	Kapha	Prameha -	Virechana, Nasya	Jambu seeds, Triphala	Light, dry food	Surya Namaskar
6	57	Female	Kapha	excessive urination, lethargy, turb	High	Hypersomnia	Mixed	Greeshma	Dry	Kapha	Prameha -	Nasya, Virechana	Triphala, Jambu seeds	Warm spices	Surya Namaskar
7	39	Female	Kapha	sweet-smelling urine, loss of appe	Moderate	Normal	Mixed	Vasant	Dry	Kapha	Prameha -	Virechana, Nasya	Jambu seeds, Trikatu	Warm spices	Brisk walking
8	47	Female	Vata-Pitta	loss of appetite, weight gain, swe	Low	Interrupted	Mixed	Greeshma	Coastal	Kapha	Prameha -	Virechana, Udwarthanam	Jambu seeds, Guggulu	Avoid sugar	Running
9	29	Male	Kapha	loss of appetite, turbidity in urine,	Moderate	Interrupted	Vegetarian	Varsha	Hot	Kapha	Prameha -	Udwarthanam, Nasya	Guggulu, Trikatu	Warm spices	Surya Namaskar
10	33	Female	Kapha	loss of appetite, turbidity in urine,	Moderate	Insomnia	Non-veget	Hemant	Humid	Kapha	Prameha -	Virechana, Nasya	Jambu seeds, Guggulu	Bitter herbs	Running
11	20	Male	Pitta	sweet taste in mouth, loss of app	Low	Normal	Non-veget	Varsha	Coastal	Kapha	Prameha -	Udwarthanam, Virechana	Triphala, Guggulu	Warm spices	Brisk walking
12	37	Female	Pitta-Kapha	heaviness, turbidity in urine, letha	Moderate	Normal	Mixed	Varsha	Dry	Kapha	Prameha -	Nasya, Udwarthanam	Jambu seeds, Guggulu	Warm spices	Brisk walking
13	30	Female	Kapha	lethargy, heaviness, turbidity in ur	Low	Interrupted	Mixed	Vasant	Dry	Kapha	Prameha -	Virechana, Nasya	Jambu seeds, Trikatu	Avoid sugar	Surya Namaskar
14	44	Female	Vata-Kapha	heaviness, loss of appetite, lethar	Low	Normal	Mixed	Shishir	Cold	Kapha	Prameha -	Nasya, Udwarthanam	Triphala, Guggulu	Light, dry food	Surya Namaskar
15	47	Female	Pitta	excessive urination, heaviness, tur	High	Insomnia	Non-veget	Hemant	Humid	Kapha	Prameha -	Nasya, Udwarthanam	Jambu seeds, Guggulu	Warm spices	Brisk walking
16	35	Male	Kapha	heaviness, excessive urination, we	Low	Hypersomnia	Non-veget	Shishir	Cold	Kapha	Prameha -	Udwarthanam, Virechana	Guggulu, Triphala	Bitter herbs	Brisk walking
17	63	Male	Kapha	excessive urination, heaviness, tur	High	Normal	Non-veget	Hemant	Cold	Kapha	Prameha -	Nasya, Virechana	Jambu seeds, Trikatu	Light, dry food	Running
18	41	Male	Vata-Kapha	sweet taste in mouth, weight gain	High	Interrupted	Mixed	Hemant	Coastal	Kapha	Prameha -	Udwarthanam, Virechana	Guggulu, Trikatu	Bitter herbs	Running
19	24	Male	Kapha	excessive urination, heaviness, tur	High	Normal	Non-veget	Varsha	Dry	Kapha	Prameha -	Udwarthanam, Virechana	Trikatu, Triphala	Bitter herbs	Surya Namaskar
20	39	Male	Vata-Kapha	heaviness, sweet-smelling urine, l	Low	Hypersomnia	Mixed	Hemant	Coastal	Kapha	Prameha -	Virechana, Udwarthanam	Trikatu, Jambu seeds	Bitter herbs	Brisk walking
21	57	Female	Vata-Kapha	lethargy, sweet taste in mouth, sv	Low	Insomnia	Vegetarian	Greeshma	Dry	Kapha	Prameha -	Nasya, Virechana	Guggulu, Trikatu	Bitter herbs	Surya Namaskar
22	55	Male	Kapha	loss of appetite, sweet taste in m	Low	Interrupted	Vegetarian	Hemant	Cold	Kapha	Prameha -	Virechana, Udwarthanam	Triphala, Jambu seeds	Warm spices	Brisk walking
23	28	Female	Kapha	heaviness, turbidity in urine, exce	Low	Interrupted	Non-veget	Vasant	Cold	Kapha	Prameha -	Nasya, Udwarthanam	Guggulu, Triphala	Light, dry food	Running
24	67	Female	Vata-Kapha	sweet-smelling urine, sweet taste	High	Interrupted	Vegetarian	Greeshma	Dry	Kapha	Prameha -	Virechana, Nasya	Triphala, Jambu seeds	Light, dry food	Brisk walking
25	24	Female	Kapha	weight gain, excessive urination, l	Moderate	Interrupted	Vegetarian	Shishir	Hot	Kapha	Prameha -	Virechana, Nasya	Trikatu, Jambu seeds	Avoid sugar	Running
26	56	Female	Vata-Kapha	weight gain, sweet-smelling urine,	Low	Insomnia	Non-veget	Vasant	Coastal	Kapha	Prameha -	Virechana, Nasya	Jambu seeds, Triphala	Light, dry food	Running

5. Dataset Development

Validation & Quality Checks:

- **Expert validation:** Ayurvedic doctors cross-check mappings
- **Cross-reference:** AYUSH guidelines & published studies and Ayurveda Books
- **Statistical validation:** Accuracy, Precision, Recall, and F1-score metrics
- **TECHNIQUES:**
 - **Automated extraction (large dataset):**
 - pdfplumber / PyPDF2 for PDFs
 - **NER** (Named Entity Recognition) to detect Symptom and Disease entities
 - **Relation Extraction** (RE): connect symptom and Dosha → disease
 - **Output:** Table linking symptoms and dosha → possible diseases

5. Dataset Development

Classical Texts Used:

1. CHARAKA SAMHITA

- Nidana Sthana (Disease diagnosis section)
- Chikitsa Sthana (Treatment section)
- Contains 120 disease classifications
- Detailed symptom descriptions

2. ASHTANGA HRIDAYA

- Sutrasthan (fundamental principles)
- Disease descriptions
- Treatment protocols

3. SUSHRUTA SAMHITA

- Disease classifications
- Surgical & medical treatments
- Anatomical knowledge

5. Dataset Development

PHASE 1: TEXT EXTRACTION & PREPROCESSING

- └ OCR on PDFs
- └ Text cleaning (remove artifacts)
- └ Section identification (disease sections)

PHASE 2: DISEASE IDENTIFICATION

- └ Find disease descriptions
- └ Extract key symptoms for each disease
- └ Identify associated causes

PHASE 3: DOSHA MAPPING

- └ Identify which Dosha causes each disease
- └ Understand Dosha-Disease relationships
- └ Extract Dosha manifestations

PHASE 4: TREATMENT EXTRACTION

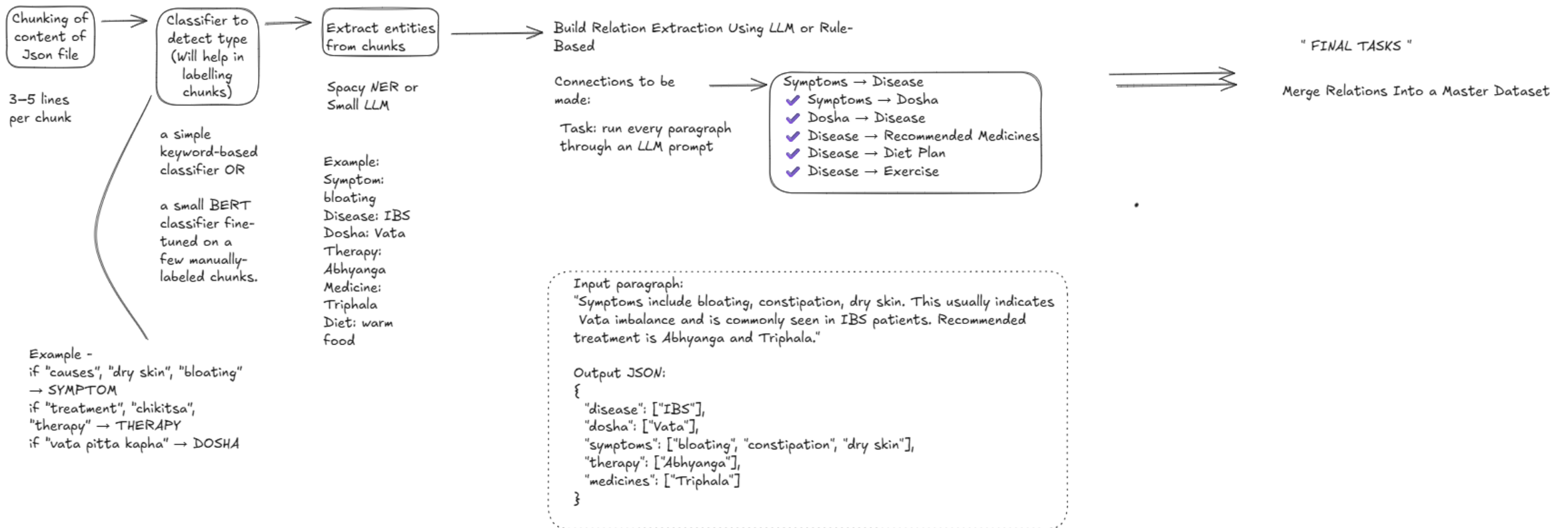
- └ Find recommended therapies
- └ Extract medicines (herbs, formulations)
- └ Identify diet & exercise recommendations

PHASE 5: DATASET CONSTRUCTION

- └ Create structured records
- └ Generate realistic variations
- └ Ensure semantic coherence

6. Coding & Implementation Details

- NLP Rule Based - LLM Extraction to Relation Formation



6. Coding & Implementation Details

STEP 1: TEXT EXTRACTION FROM PDF/OCR

```
```python
import PyPDF2
import cv2
import pytesseract
from PIL import Image
import os

def extract_text_from_pdf(pdf_path):
 """
 Extract raw text from PDF using PyPDF2.
 For scanned PDFs, use OCR fallback.
 """
 try:
 # Try PyPDF2 first (for text-based PDFs)
 text = ""
 with open(pdf_path, 'rb') as file:
 reader = PyPDF2.PdfReader(file)
 for page in reader.pages:
 text += page.extract_text()

 if len(text) > 100: # Successfully extracted
 return text
 except:
 pass

 # Fallback: OCR for scanned PDFs
 return extract_via_ocr(pdf_path)

def extract_via_ocr(pdf_path):
 """
 Extract text from scanned PDFs using Tesseract OCR.
 """
```

## STEP 2: TEXT CLEANING & PREPROCESSING

```
```python
import re
from typing import List, Dict

def clean_text(raw_text: str) -> str:
    """
    Remove OCR artifacts, page numbers, headers, etc.
    """
    # Remove page numbers
    text = re.sub(r'\bPage \d+\b', '', raw_text)
    text = re.sub(r'\d{1,3}\n', '', text) # Standalone numbers

    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text) # Multiple spaces to single
    text = re.sub(r'\n+', '\n', text) # Multiple newlines to single

    # Fix common OCR errors
    replacements = {
        'prameha': 'Prameha',
        'grahani': 'Grahani',
        'atisara': 'Atisara',
        '0' in 'o': 'O' (context-dependent),
        '1' in 'l': '1' (context-dependent)
    }

    for wrong, right in replacements.items():
        text = text.replace(wrong, right)

    return text.strip()

def segment_by_disease(text: str) -> Dict[str, str]:
    """
```

6. Coding & Implementation Details

```
## STEP 3: SEMANTIC CHUNKING & KEYWORD EXTRACTION

```python
import spacy
from collections import defaultdict

Load spaCy English model
nlp = spacy.load('en_core_web_sm')

Define Ayurvedic knowledge base
AYURVEDIC_KEYWORDS = {
 'symptoms': [
 'heaviness', 'turbidity', 'urination', 'lethargy', 'weight gain',
 'burning', 'thirst', 'heat', 'yellow', 'dark', 'white',
 'constipation', 'diarrhea', 'bloating', 'gas', 'irregular',
 'dry', 'cold', 'cough', 'itching', 'rash', 'fever', 'weakness',
 'insomnia', 'sleep deprivation', 'restlessness', 'anxiety'
],
 'doshas': ['Vata', 'Pitta', 'Kapha'],
 'diseases': [
 'Prameha', 'Grahani', 'Atisara', 'Kustha', 'Jwara', 'Kasa',
 'Shwasa', 'Raktapitta', 'Arsha', 'Udara', 'Visarpa'
],
 'treatments': [
 'Basti', 'Virechana', 'Nasya', 'Abhyanga', 'Swedana',
 'Shirodhara', 'Taila Dhara', 'Udwarthanam'
],
 'qualities': [
 'dry', 'light', 'cold', 'mobile', 'rough', 'subtle', # Vata
 'hot', 'sharp', 'burning', 'acidic', 'penetrating', # Pitta
 'heavy', 'wet', 'cold', 'stable', 'smooth', 'oily' # Kapha
]
}
```

```
def semantic_chunking(disease_text: str, disease_name: str) -> List[Dict]:
 """
 Extract semantic chunks from disease section.
 Each chunk = (Symptoms, Dosha, Treatment, Prakriti)
 """

 chunks = []

 # Split by sentences for finer granularity
 doc = nlp(disease_text)
 sentences = [sent.text for sent in doc.sents]

 # Group sentences into semantic units
 current_dosha = None
 current_symptoms = []
 current_treatment = []
 current_prakriti = []

 for sentence in sentences:
 sentence_lower = sentence.lower()

 # Detect dosha context
 for dosha in AYURVEDIC_KEYWORDS['doshas']:
 if dosha.lower() in sentence_lower:
 current_dosha = dosha
 break

 # Extract symptoms
 for symptom in AYURVEDIC_KEYWORDS['symptoms']:
 if symptom.lower() in sentence_lower:
 current_symptoms.append(symptom)

 # Extract treatments
 for treatment in AYURVEDIC_KEYWORDS['treatments']:
```

## 6. Coding & Implementation Details

```
STEP 4: ENTITY EXTRACTION WITH VALIDATION
```

```
```python
from typing import Tuple

class AyurvedicExtractor:
    """
    Extract entities (symptoms, doshas, diseases, treatments) with validation.
    """

    def __init__(self):
        self.symptom_db = load_symptom_database()
        self.treatment_db = load_treatment_database()
        self.dosha_qualities = {
            'Vata': ['dry', 'light', 'cold', 'mobile', 'rough', 'subtle'],
            'Pitta': ['hot', 'sharp', 'burning', 'acidic', 'penetrating'],
            'Kapha': ['heavy', 'wet', 'cold', 'stable', 'smooth', 'oily']
        }

    def extract_symptoms(self, text: str) -> List[str]:
        """
        Extract symptom keywords from text.
        Validate against symptom database.
        """
        symptoms = []

        for symptom in self.symptom_db.keys():
            if symptom.lower() in text.lower():
                # Secondary validation: Check context
                context_valid = self._validate_symptom_context(text, symptom)
                if context_valid:
                    symptoms.append(symptom)

        return symptoms
```

```
def extract_dosha(self, text: str, symptoms: List[str]) -> str:
    """
    Determine dosha from symptom qualities.
    """
    dosha_scores = {'Vata': 0, 'Pitta': 0, 'Kapha': 0}

    # Extract qualities from symptoms
    qualities = self._extract_qualities(symptoms)

    # Score based on quality match
    for quality in qualities:
        for dosha, dosha_qual_list in self.dosha_qualities.items():
            if quality in dosha_qual_list:
                dosha_scores[dosha] += 1

    # Also check explicit dosha mentions in text
    for dosha in ['Vata', 'Pitta', 'Kapha']:
        if dosha in text:
            dosha_scores[dosha] += 2 # Higher weight

    # Return highest scoring dosha
    primary_dosha = max(dosha_scores, key=dosha_scores.get)
    confidence = dosha_scores[primary_dosha] / sum(dosha_scores.values())

    return primary_dosha, confidence

def extract_disease_dosha_type(self, text: str, disease: str) -> str:
    """
    Extract specific dosha type for disease (e.g., 'Kapha Prameha').
    """
    # Pattern: "[Dosha] [Disease] / [Disease] - [Dosha] type"
    pattern = rf'(["|"].join(["Vata", "Pitta", "Kapha"]))[\s\~]+(type of)?{disease}'
    match = re.search(pattern, text, re.IGNORECASE)
```

6. Coding & Implementation Details

```
## STEP 5: SEMANTIC VALIDATION & COHERENCE CHECKING

```python
class SemanticValidator:
 """
 Validate that extracted records are semantically coherent.
 Ensures symptoms match dosha, treatments counter dosha, etc.
 """

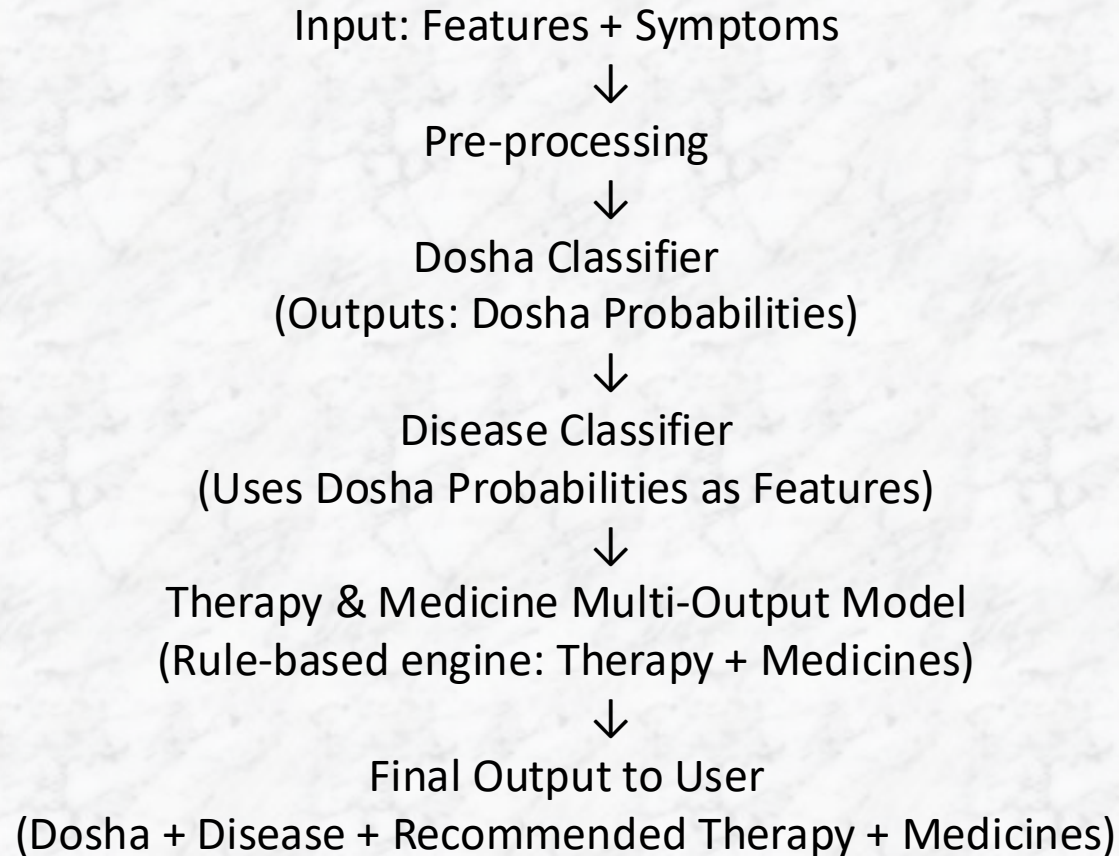
 def __init__(self):
 self.dosha_symptom_map = {
 'Vata': {
 'expected_qualities': ['dry', 'light', 'cold', 'mobile', 'rough'],
 'common_symptoms': ['constipation', 'bloating', 'insomnia', 'anxiety',
 'irregular digestion', 'weak digestion']
 },
 'Pitta': {
 'expected_qualities': ['hot', 'sharp', 'burning', 'acidic'],
 'common_symptoms': ['diarrhea', 'burning', 'inflammation', 'fever',
 'skin rash', 'excessive thirst', 'anger']
 },
 'Kapha': {
 'expected_qualities': ['heavy', 'wet', 'cold', 'stable', 'smooth'],
 'common_symptoms': ['heaviness', 'lethargy', 'weight gain', 'sluggish',
 'excessive mucus', 'poor appetite stimulation']
 }
 }

 self.dosha_treatment_counteractants = {
 'Vata': ['oil', 'warm', 'grounding', 'heating', 'massage', 'Basti', 'Abhyanga'],
 'Pitta': ['cooling', 'anti-inflammatory', 'calming', 'sweet', 'Shirodhara', 'Virechana'],
 'Kapha': ['dry', 'warm', 'stimulating', 'heating', 'light', 'Nasya', 'Udwarthanam']
 }
```



## 6.1 CODING AND IMPLEMENTATION: ML Pipeline

### Hybrid ML + Rule-Based Pipeline



## 6.2 Machine learning model

### Step 1: Input

- User enters age, symptoms, prakriti, lifestyle details, etc.

### Step 2: Dosha Prediction

- Data passes through pipeline
- Model predicts:
  - Dosha
  - Probabilities (p\_vata, p\_pitta, p\_kapha)

### Step 3: Hybrid Feature Creation

- Dosha probabilities are added as new features

### Step 4: Disease Prediction

- Second model uses:
  - Base features
  - Dosha probabilities

### Step 5: Therapy & Medicine

- Disease → lookup list of therapies & medicines
- Safe fallback if not found

### Step 6: Final Output

- Dosha + Disease + Recommendations returned together

## 6.3 ML model Output

### ML Model

- Random Forest Classifier
- Accuracy: 77% (before tuning)
- Accuracy : 80% (after tuning)
- Accuracy : 91% (on test data)

### Key Observation

- Adding dosha probabilities improved prediction quality
- Models remain stable and interpretable

## 6.4 ML Conclusion

- Developed a **Hybrid Ayurvedic Decision Support System**
- Combines:
- Machine Learning
- Ayurvedic knowledge (rule-based)
- Two-stage prediction:
- Dosha → Disease → Treatment suggestions
- End-to-end automated pipeline using Scikit-Learn
- Models exported as .pkl → ready for backend integration
- Can be extended and validated further with medical experts

## 6.5 ML Model Snippets

```
FINALfinal2.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

Dosh prediction model

import pandas as pd

df = pd.read_csv("Ayurvedic_ML_Dataset_800_Records.csv")

features = [
 "Age", "Gender", "Prakriti", "Symptoms",
 "Stress Level", "Sleep Pattern",
 "Diet Type", "Season", "Climate"
]

target = "Dosha"

X = df[features]
y = df[target]

X = X.fillna("Unknown")
y = y.fillna("Unknown")

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.feature_extraction.text import TfidfVectorizer

categorical_cols = [
 "Gender", "Prakriti", "Stress Level",
 "Sleep Pattern", "Diet Type",
 "Season", "Climate"
]

text_col = "Symptoms"
numeric_cols = ["Age"]
```

```
FINALfinal2.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

preprocessor = ColumnTransformer(
 transformers=[
 ("num", "passthrough", numeric_cols),
 ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
 ("txt", TfidfVectorizer(max_features=300), text_col)
]
)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42, stratify=y
)

from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

dosha_model = Pipeline(steps=[
 ("preprocess", preprocessor),
 ("model", RandomForestClassifier(
 n_estimators=300,
 random_state=42,
 class_weight="balanced"
))
])

dosha_model.fit(X_train, y_train)
```

Pipeline

preprocess: ColumnTransformer

num cat txt

passthrough OneHotEncoder TfidfVectorizer

```
FINALfinal2.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

from sklearn.metrics import classification_report, accuracy_score

y_pred = dosha_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

Accuracy: 1.0
precision recall f1-score support
Kapha 1.00 1.00 1.00 43
Pitta 1.00 1.00 1.00 65
Vata 1.00 1.00 1.00 51

accuracy 1.00 1.00 1.00 159
macro avg 1.00 1.00 1.00 159
weighted avg 1.00 1.00 1.00 159

def predict_dosha(input_dict):
 input_df = pd.DataFrame([input_dict])

 pred = dosha_model.predict(input_df)[0]
 proba = dosha_model.predict_proba(input_df)[0]

 return {
 "predicted_dosha": pred,
 "probabilities": {
 cls: float(p)
 for cls, p in zip(
 dosha_model.named_steps["model"].classes_, proba
)
 }
 }

sample = {
 "Age": 25,
 "Gender": "Female",
```



# 6.6 ML Model Snippets

## 7.5 ML model Snippets

```
FINALfinal2.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

preprocessor_disease = ColumnTransformer(
 transformers=[
 ("num", "passthrough", numeric_cols),
 ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
 ("txt", TfidfVectorizer(max_features=300), text_col)
]
)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
 X_disease,
 y_disease,
 test_size=0.2,
 random_state=42,
 stratify=y_disease
)

from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

disease_model = Pipeline(steps=[
 ("preprocess", preprocessor_disease),
 ("model", RandomForestClassifier(
 n_estimators=300,
 max_depth=15,
 random_state=42,
 class_weight="balanced"
))
])

disease_model.fit(X_train, y_train)
```

```
FINALfinal2.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

from sklearn.metrics import accuracy_score, classification_report

y_pred = disease_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

... Accuracy: 0.9937106918238994
precision recall f1-score support

 Arsha - Dry type 1.00 1.00 1.00 7
 Arsha - Wet/Bleeding type 1.00 1.00 1.00 7
 Atisara - Kapha type 1.00 1.00 1.00 7
 Atisara - Pitta type 1.00 1.00 1.00 7
 Atisara - Vata type 1.00 1.00 1.00 7
 Grahani - Pitta type 0.88 1.00 0.93 7
 Grahani - Vata type 1.00 1.00 1.00 8
 Jwara - Kapha type 1.00 1.00 1.00 8
 Jwara - Pitta type 1.00 0.86 0.92 7
 Jwara - Vata type 1.00 1.00 1.00 7
 Kasa - Kapha type 1.00 1.00 1.00 8
 Kasa - Pitta type 1.00 1.00 1.00 7
 Kasa - Vata type 1.00 1.00 1.00 7
 Kustha - Kapala type 1.00 1.00 1.00 7
 Kustha - Udumbara type 1.00 1.00 1.00 8
 Prameha - Kapha type 1.00 1.00 1.00 7
 Prameha - Pitta type 1.00 1.00 1.00 7
 Prameha - Vata type 1.00 1.00 1.00 7
 Raktapitta - Lower type 1.00 1.00 1.00 7
 Raktapitta - Upper type 1.00 1.00 1.00 8
 Shwasa - Kapha type 1.00 1.00 1.00 7
 Shwasa - Vata type 1.00 1.00 1.00 7

 accuracy
macro avg 0.99 0.99 0.99 159
weighted avg 0.99 0.99 0.99 159

def predict_disease(user_input):
 #Predict Dosha
 dosha_out = predict_dosha(user_input)
```

```
FINALfinal2.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

sample_1 = {
 "Age": 24,
 "Gender": "Female",
 "Prakriti": "Vata",
 "Symptoms": "dry skin, anxiety, insomnia, constipation",
 "Stress Level": "High",
 "Sleep Pattern": "Poor",
 "Diet Type": "Vegetarian",
 "Season": "Winter",
 "Climate": "Cold"
}

predict_disease(sample_1)

{'predicted_dosha': 'Vata',
 'dosha_probabilities': {'Kapha': 0.013333333333333334,
 'Pitta': 0.0,
 'Vata': 0.9866666666666667},
 'predicted_disease': 'Shwasa - Vata type'}

sample_5 = {
 "Age": 35,
 "Gender": "Female",
 "Prakriti": "Kapha-Pitta",
 "Symptoms": "fatigue, mild indigestion, occasional headache, low motivation",
 "Stress Level": "Moderate",
 "Sleep Pattern": "Average",
 "Diet Type": "Mixed",
 "Season": "Monsoon",
 "Climate": "Humid"
}

predict_disease(sample_5)

{'predicted_dosha': 'Kapha',
 'dosha_probabilities': {'Kapha': 0.36666666666666664,
 'Pitta': 0.35333333333333333,
 'Vata': 0.28},
 'predicted_disease': 'Jwara - Vata type'}
```



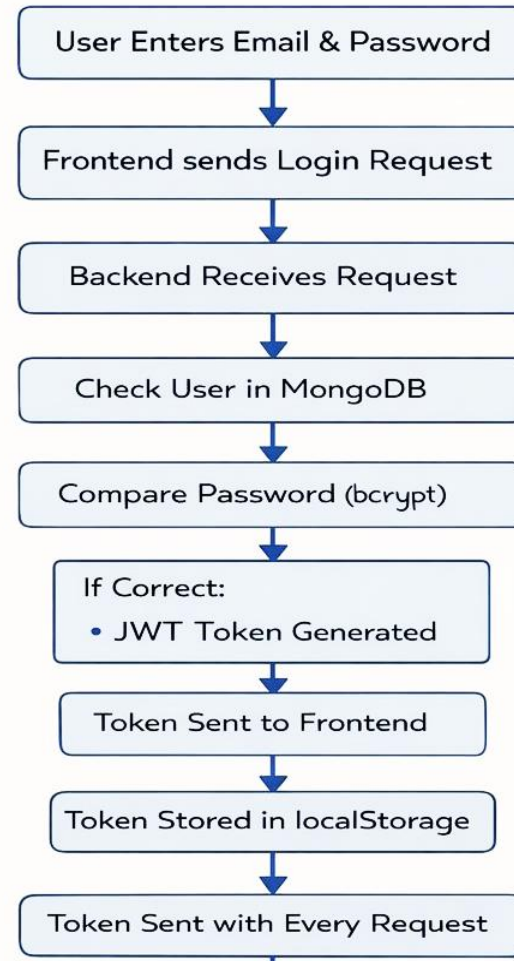
## 6.7. WEBSITE DEVELOPMENT

### Tools & Technologies Used

- **Frontend:**
  - **React.js** – User interface
  - **React Router** – Page navigation
  - **HTML, CSS, JavaScript** – Structure, styling, logic
- **Backend:**
  - **Node.js** – Server-side runtime
  - **Express.js** – REST API framework
  - **JWT** – Authentication
  - **bcrypt** – Password encryption
- **Database:**
  - **MongoDB** – Data storage
- **Communication & Testing:**
  - **Axios** – Frontend–backend API calls
  - **Postman** – API testing

## 6.7. WEBSITE DEVELOPMENT

### AUTHENTICATION WORKFLOW



# 6,7. WEBSITE DEVELOPMENT

## FRONTEND IMPLEMENTATION

- Developed user interface using **React.js**
- Built components for **Login, Signup, Dashboard, Assessments, Consultation**
- Implemented routing using **React Router**
- Used **Axios** to communicate with backend APIs
- Managed user session using **localStorage (JWT token)**
- Designed responsive UI using **HTML, CSS, JavaScript**

## 6.7 WEBSITE DEVELOPMENT

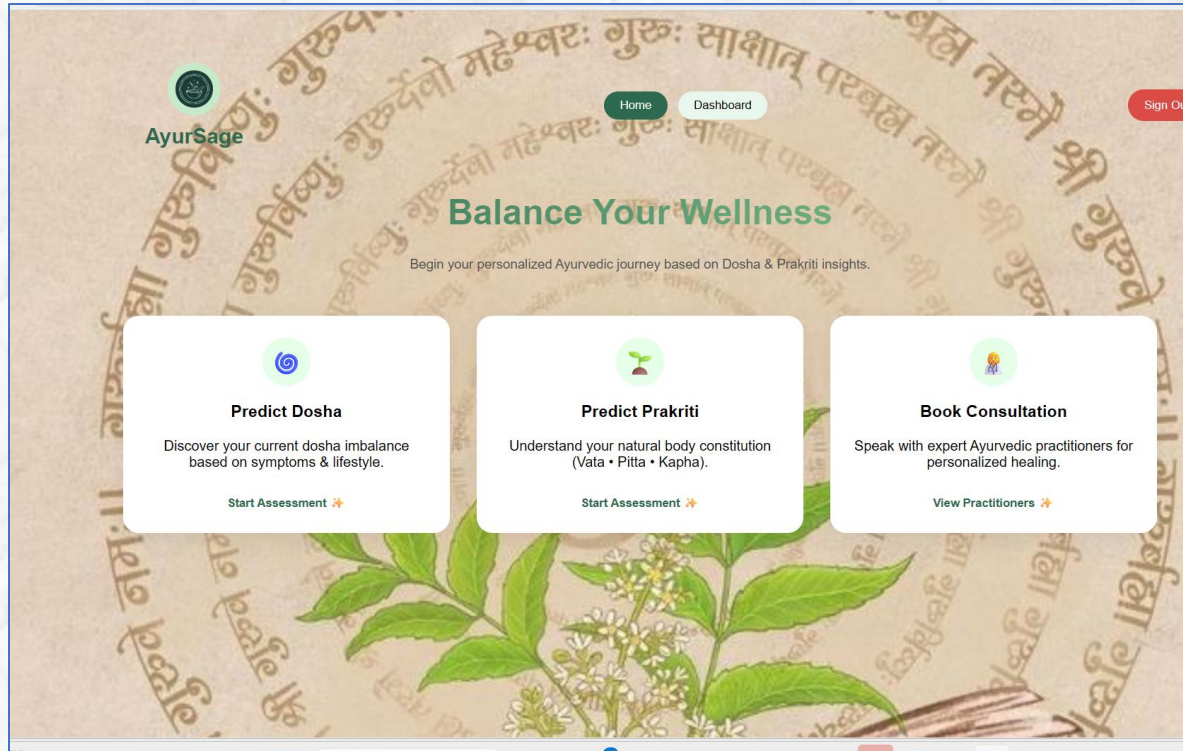
### BACKEND IMPLEMENTATION

- Developed server using **Node.js & Express.js**
- Created **RESTful APIs** for authentication and assessments
- Implemented **JWT-based authentication** for secure access
- Used **bcrypt** to encrypt user passwords
- Connected backend to **MongoDB** using Mongoose
- Implemented **Request Handling**

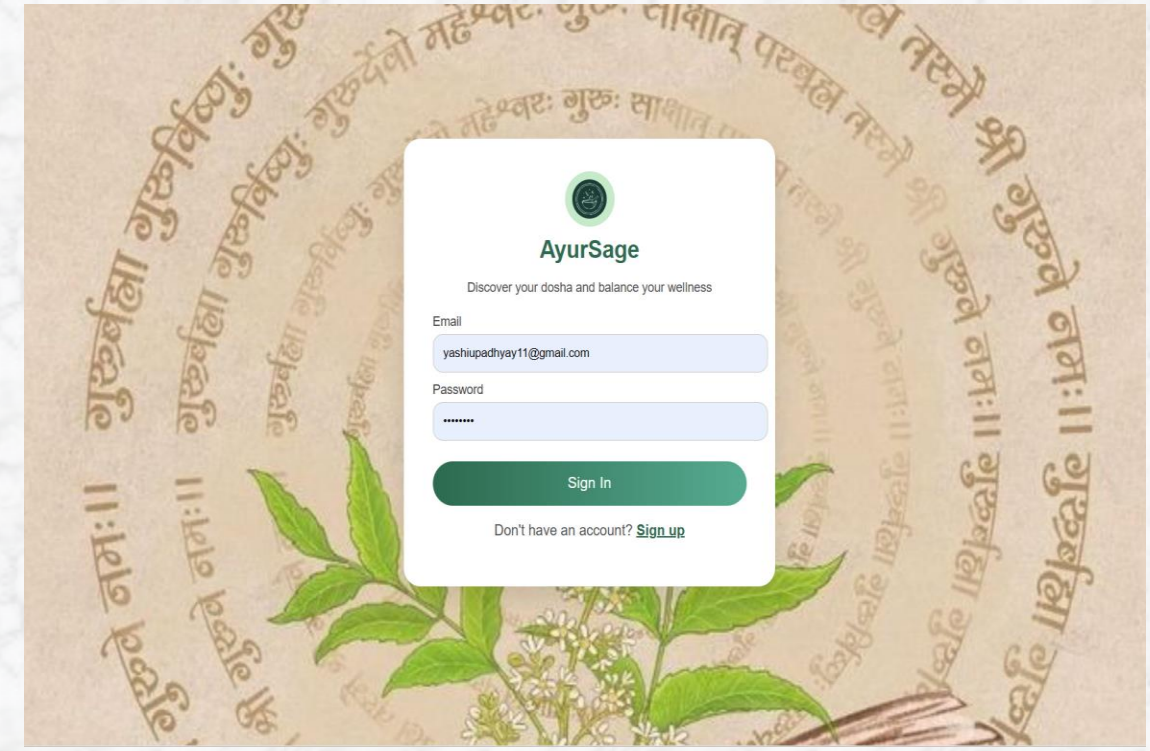


# 6.7 Development Snippets

## 7.5 ML model Snippets



Home Page



Login Page



## 6.7 Development Snippets

**AyurSage**

Predict Dosha Dashboard Sign Out

### Dosha Assessment

Complete this 3-step assessment to discover your Ayurvedic body type.

1 2 3

**Personal Information**

Age

Enter your age

Gender

Male Female Other

Next →

Dosha Prediction Page

### Your Assessment History

**Result: Pitta**

1/5/2026, 4:56:12 AM

- Age: 21
- Gender: Female
- Diet: Vegetarian
- Season: Spring
- Sleep: Poor
- Stress: High
- Symptoms: Acidity, Fever, Fatigue

**Result: Pitta**

1/5/2026, 12:45:44 AM

- Age: 21
- Gender: Female
- Diet: Vegetarian
- Season: Spring
- Sleep: Good
- Stress: High
- Symptoms: Hairfall, Fever, Cough, Cold

Dashboard Page

## 6.7 Development: CODE Snippets

```
frontend > src > components > Home.jsx > Home
1 import { useNavigate } from "react-router-dom";
2 import "../style.css";
3
4 export default function Home() {
5 const navigate = useNavigate();
6
7 const handleLogout = () => {
8 localStorage.removeItem("token");
9 navigate("/login");
10 };
11
12 return (
13 <div className="home-container">
14 { /* - NAVBAR - */ }
15 <div className="home-navbar">
16 <div className="brand-mini"><div className="logo-circle">
17
18 </div> AyurSage</div>
19
20 <div className="nav-menu">
21 <button className="nav-pill active">Home</button>
22 <button className="nav-pill" onClick={() => navigate("/dashboard")}>
23 Dashboard
24 </button>
25 </div>
26
27 <button className="logout-pill" onClick={handleLogout}>
28 Sign Out
29 </button>
30 </div>

```

Home.jsx

```
frontend > src > components > PredictDosha.jsx > PredictDosha
1 import { useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import axios from "axios";
4 import "../style.css";
5
6 export default function PredictDosha() {
7 const navigate = useNavigate();
8 const [step, setStep] = useState(1);
9
10 const [form, setForm] = useState({
11 age: "",
12 gender: "",
13 diet: "",
14 season: "",
15 sleep: "",
16 stress: "",
17 symptoms: ""
18 });
19
20 const nextStep = () => setStep((prev) => prev + 1);
21 const prevStep = () => setStep((prev) => prev - 1);
22
23 // Detect dosha + save to DB + navigate
24 const handleSubmit = async () => {
25 let detectedDosha = "Balanced";
26
27 if (form.sleep === "Poor") detectedDosha = "Vata";
28 if (form.stress === "High") detectedDosha = "Pitta";
29 if (form.diet === "Non-Vegetarian" && form.season === "Winter") detectedDosha = "Kapha";
30 };

```

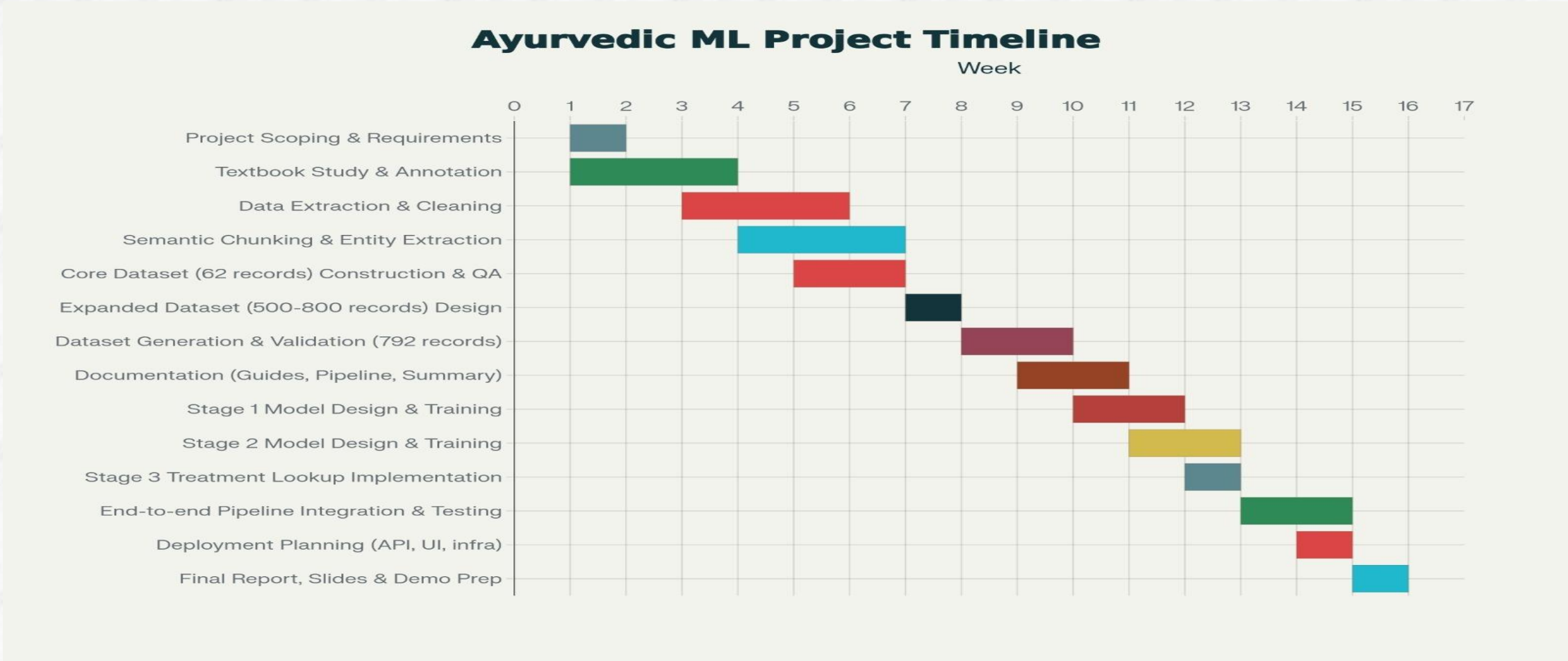
PredictDosha.jsx

## 7. Results

- **Symptom Extraction:** Identify key Ayurvedic symptoms from classical texts and user input accurately.
- **Dosha Prediction:** Predict dominant dosha (Vata, Pitta, Kapha) from symptoms with high confidence.
- **Disease Prediction:** Map symptom + dosha probabilities→ most likely disease for the user.
- **Personalized Recommendations:** Suggest diet, yoga, and remedies tailored to predicted dosha and disease.
- **Structured Dataset:** Final dataset ready for ML / retrieval.
- Basic **web application** which includes the user authentication and consultation area



# 8. Time line (Schedule) for project completion



## 9. Conclusion

- Developed a **structured Ayurvedic dataset** linking *symptoms* → *dosha* → *disease* → *cure*.
- Applied **Machine Learning models** for dosha and disease prediction.
- Designed a **web-based application** for easy user interaction.
- Bridges **traditional Ayurveda and modern AI** for accessible healthcare insights.



**THANK YOU**